You said:

codigo.txt
Document

Analise o arquivo em anexo passo a passo e determine ao final: este arquivo é benigno ou é um malware?

Worked for 21s

ChatGPT said:

Análise de segurança do arquivo `codigo.txt`

MALWARE — CONFIRMADO

Classificação principal: Ransomware (Crypto-Ransomware / Ransomware de criptografia de arquivos).

O código apresenta comportamento malicioso explícito: percorre unidades de armazenamento, criptografa arquivos pessoais, apaga os arquivos originais e deixa mensagens exigindo pagamento em Bitcoin para supostamente recuperar os dados.

A seguir, a análise está dividida em etapas, considerando os quatro blocos de código apresentados no arquivo.

1. Análise do Código 1 — Criação de arquivos auxiliares

O primeiro bloco contém a classe `crea` e o método `inputStreamToFile()`.

```
final OutputStream os = new FileOutputStream(file);
while ((bytesRead = inputStream.read(buffer, 0, 8192)) != -1) {
    os.write(buffer, 0, bytesRead);
}
```

O método copia o conteúdo de um fluxo de entrada para um arquivo no sistema.

O método `creat()` utiliza essa funcionalidade para extrair um recurso chamado `JavaFX.jar` e salvá-lo com o nome:

```
Javalockerencrypt.jar
```

O arquivo é colocado no diretório recebido como parâmetro.

Avaliação: esse comportamento é suspeito porque o programa deposita um arquivo auxiliar em diretórios do sistema de arquivos. Entretanto, o conteúdo do JAR não está disponível para análise, portanto não é possível determinar suas funcionalidades ou afirmar que ele próprio seja malicioso. codigo.txt

2. Análise do Código 2 — Funções de hash MD5

O segundo bloco implementa uma classe chamada `Encryption`.

Ela utiliza:

```
MessageDigest md = MessageDigest.getInstance("MD5");
byte[] results = md.digest(originstr.getBytes());
```

A finalidade aparente é produzir um hash MD5 de uma string, possivelmente para gerar ou verificar senhas.

Existe, inclusive, uma função de autenticação:

```
public static boolean authenticatePassword(String pass, String inputstr) {
    return pass.equals(encodeByMD5(inputstr));
}
```

Entretanto, há um problema na conversão dos bytes para hexadecimal:

```
int d1 = n / 16;
int d2 = n / 16;
```

O cálculo de `d2` repete o cálculo de `d1`, quando normalmente deveria utilizar o resto da divisão por 16. Isso produz uma representação hexadecimal incorreta.

Além disso, essa classe não é responsável pela criptografia dos arquivos executada pelo programa principal.

Avaliação: código relacionado ao processamento de senhas, com uma implementação defeituosa. Isoladamente, não caracteriza malware. codigo.txt

3. Análise do Código 3 — Identificação dos arquivos que serão atacados

Este é um dos blocos mais importantes da análise.

O método `tryencrpt()` contém uma extensa lista de extensões de arquivos:

```
final String[] type = {
    ".accdb", ".doc", ".docx", ".xls", ".xlsx",
    ".pdf", ".txt", ".csv", ".zip", ".7z",
    ".rar", ".jpg", ".jpeg", ".png", ".mp4",
    ".mp3", ".sql", ".sqlite3", ".pst", ".db"
};
```

A lista inclui documentos, imagens, vídeos, arquivos compactados, bancos de dados, arquivos de projetos e outros tipos de dados pessoais.

O código verifica a extensão de cada arquivo. Quando encontra uma extensão correspondente, executa:

```
key.main(p, p + ".javalocker");
```

Isso chama a rotina de criptografia do Código 4, criando uma versão criptografada com o sufixo:

```
.javalocker
```

Em seguida, executa:

```
final File f = new File(p);
if (f.canWrite()) {
    f.delete();
}
```

Esse comportamento é particularmente perigoso: o arquivo original é apagado depois da tentativa de criptografia.

O código não verifica se a criptografia foi concluída com sucesso antes de apagar o original. Além disso, eventuais exceções são ignoradas.

Avaliação: comportamento diretamente associado a ransomware. O malware procura arquivos valiosos e substitui sua disponibilidade por versões criptografadas. codigo.txt

4. Análise do Código 3 — Disseminação do ataque pelos diretórios

Os métodos `folderMethod1()` e `find2()` são responsáveis pela procura de arquivos e diretórios.

O método `folderMethod1()` utiliza uma lista para percorrer recursivamente a estrutura de diretórios:

```
list.addAll(Arrays.asList(file.listFiles()));

while (!list.isEmpty()) {
    final File[] files = list.removeFirst().listFiles();
```

Para cada diretório encontrado, o programa continua a busca.

Para arquivos menores que 100 MB, chama o método de criptografia:

```
if (f.length() < 104857600L) {
    tryencrpt(f.getAbsolutePath());
}
```

Isso demonstra uma tentativa de automatizar a criptografia em grande quantidade de arquivos.

O programa exclui explicitamente diretórios cujo caminho começa com `C:\Windows` nessa rotina específica. Contudo, essa proteção não é aplicada de maneira uniforme em `find2()`.

Avaliação: a busca recursiva é uma característica importante do comportamento de um ransomware, pois permite atingir arquivos em diversos diretórios, em vez de operar somente sobre um arquivo fornecido manualmente. codigo.txt

5. Análise do Código 3 — Criação da mensagem de resgate

Um dos indícios mais conclusivos está na criação de um arquivo chamado:

```
readmeonnotepad.javaencrypt
```

O conteúdo gravado nele inclui:

```
Your personal files are encrypted by javalocker!

You need to send 300$ of bitcoins to the following adress:
BAW4VM2dhxYgXeQepOHKHSQVG6NgaEb94

then contact soviet@12334@gmail.com!
```

A mensagem afirma expressamente que os arquivos pessoais foram criptografados e exige US$ 300 em Bitcoin para sua recuperação.

O programa cria essa mensagem nos diretórios encontrados e também deposita o arquivo JAR auxiliar nesses locais.

Avaliação: a exigência de pagamento em troca da recuperação de arquivos é uma característica inequívoca de ransomware. codigo.txt

6. Análise do Código 4 — Rotina de criptografia DES

O quarto bloco implementa a classe `key`, responsável pela criptografia e descriptografia.

O programa inicializa um gerador de chaves DES:

```
KeyGenerator _generator = KeyGenerator.getInstance("DES");
_generator.init(new SecureRandom(strKey.getBytes()));
this.key = _generator.generateKey();
```

A chave é inicializada a partir de uma string fixa:

```
final key td = new key("dsjfvif$$#%$#jvfdsjf@$@kjvfsfh@$#shrvehdf@$#$");
```

O método `encrypt()` utiliza:

```
final Cipher cipher = Cipher.getInstance("DES");
cipher.init(1, this.key);
```

O valor `1` corresponde ao modo de criptografia (`ENCRYPT_MODE`).

Depois, o programa abre o arquivo original e grava o resultado criptografado no arquivo de destino:

```
final InputStream is = new FileInputStream(file);
final OutputStream out = new FileOutputStream(destFile);
final CipherInputStream cis = new CipherInputStream(is, cipher);
```

A rotina lê os dados em blocos e escreve o resultado criptografado.

O código também possui um método `decrypt()`, demonstrando a existência de uma rotina de descriptografia. Entretanto, não há uma interface de recuperação efetivamente apresentada no programa.

Uma observação técnica: DES é um algoritmo criptográfico antigo e considerado inseguro para aplicações modernas. Além disso, o uso de `SecureRandom` com uma semente fixa não garante, por si só, a reprodução da mesma chave em todas as implementações e ambientes Java.

Avaliação: trata-se da implementação efetiva da criptografia utilizada pelo ransomware.

codigo.txt

7. Análise do método principal — Execução do ataque

O método `main()` mostra como o ataque é iniciado.

Primeiro, o programa cria mensagens de resgate nas raízes das unidades:

```
ret(new File("C:\\"));
ret(new File("D:\\"));
ret(new File("E:\\"));
ret(new File("F:\\"));
ret(new File("G:\\"));
ret(new File("H:\\"));
```

Depois, percorre as unidades, executando os métodos de busca e criptografia:

```
find2("D:\\");
folderMethod1("D:\\");

find2("E:\\");
folderMethod1("E:\\");
```

O mesmo procedimento é repetido para as unidades C:, D:, E:, F:, G: e H:, além de uma tentativa de varredura a partir de `/`.

Isso demonstra que o programa foi projetado para atingir múltiplas unidades de armazenamento, ampliando o impacto do ataque.

codigo.txt

8. Indicadores de comprometimento identificados

Indicador	Evidência
Criptografia de arquivos	Utilização do algoritmo DES
Extensões específicas	Lista extensa de arquivos pessoais
Renomeação dos arquivos	Acréscimo de `.javalocker`
Exclusão dos originais	Chamada a `File.delete()`
Nota de resgate	`readmeonnotepad.javaencrypt`
Extorsão financeira	Exigência de US$ 300 em Bitcoin
Contato do criminoso	Endereço de e-mail na nota
Varredura de diretórios	Percurso recursivo do sistema
Ataque a várias unidades	C:, D:, E:, F:, G: e H:

9. Conclusão final

ARQUIVO MALICIOSO Malware

Categoria principal: Ransomware de criptografia de arquivos (*Crypto-Ransomware*).

Família ou identificação apresentada pelo próprio código: JavaLocker.

Nível de confiança da classificação: Muito alto.

A conclusão é que o código é **malicioso, não benigno**.

As evidências determinantes são a criptografia automatizada de arquivos, a exclusão dos originais, a criação de notas de resgate e a exigência explícita de pagamento em Bitcoin.

Não há evidência suficiente neste código para afirmar que exista roubo de dados, comunicação com servidor remoto ou mecanismo de persistência na inicialização do sistema. Esses comportamentos não são necessários, entretanto, para classificar o programa como ransomware.

Recomendação: não execute esse programa em um computador real. Caso já tenha sido executado, isole o equipamento da rede e preserve os arquivos criptografados para investigação e eventual recuperação.

You said:

Qual é a categoria principal deste malware?

ChatGPT said:

A categoria principal deste malware é **Ransomware (Crypto-Ransomware)**.

Ele criptografa arquivos da vítima usando DES, adiciona a extensão `.javalocker` aos arquivos criptografados, apaga os originais e exige pagamento de US$ 300 em Bitcoin para supostamente recuperar os dados.